Лабораторна робота 2: короткі теоретичні відомості

Цей конспект є продовженням матеріалів до ЛР №1. У першій роботі було створено базовий CRUD-каталог. У другій потрібно не просто додати поле, пошук і перевірку: студент має побачити, як **URL передає дані**, як **форма передає дані в C#**, як **LINQ перетворюється на SQL**, а зміна класу моделі потрапляє до **схеми бази даних**.

Приклади нижче використовують сутність `Book` (книга). Для каталогу товарів, музики чи адрес змінюється назва класу та його властивості, але принципи залишаються тими самими.

1. Результат лабораторної роботи

Після виконання роботи застосунок має вміти:

- показувати зрозумілі назви полів і форматувати дані;
- відкривати деталі, редагування й видалення запису за адресою з ідентифікатором;
- шукати записи за текстом;
- фільтрувати записи за категорією;
- додавати нову властивість до моделі та без втрати даних оновлювати БД міграцією;
- перевіряти дані як у браузері, так і на сервері.

2. Модель — джерело опису даних

Клас моделі описує одну сутність предметної області. У EF Core він також впливає на структуру таблиці БД, а у Razor Pages — на поля форми та відображення значень.

```
using System.ComponentModel.DataAnnotations;
using Microsoft.EntityFrameworkCore;

public class Book
{
    public int Id { get; set; }

    [Required(ErrorMessage = "Вкажіть назву")]
    [StringLength(120, MinimumLength = 2)]
    [Display(Name = "Назва")]
    public string Title { get; set; } = string.Empty;

    [Display(Name = "Дата видання")]
    [DataType(DataType.Date)]
    public DateTime PublishedDate { get; set; }

    [Required]
    [StringLength(60)]
    [Display(Name = "Категорія")]
    public string Category { get; set; } = string.Empty;
```

```
    [Range(0.01, 1_000_000, ErrorMessage = "Ціна має бути більшою за нуль")]
    [Precision(10, 2)]
    [DataType(DataType.Currency)]
    [Display(Name = "Ціна")]
    public decimal Price { get; set; }

    [StringLength(30)]
    [Display(Name = "Видавництво")]
    public string? Publisher { get; set; }
}
```

Одна властивість може мати кілька ролей:

Елемент	Що він означає
`public string Title`	Текстова властивість об'єкта `Book`.
`[Display(Name = "Назва")]`	Назва для таблиць, форм і підписів у UI. Не змінює назву стовпця БД.
`[Required]`	Значення є обов'язковим. Перевіряється під час роботи форми.
`[StringLength(120)]`	Обмежує довжину тексту; EF Core може створити відповідне обмеження стовпця.
`[DataType(DataType.Date)]`	Підказує UI, що це дата. Це форматування, а не правило валідації.
`[Precision(10, 2)]`	Налаштовує точність десяткового числа для EF Core: 10 цифр загалом, 2 після коми.

> У прикладі з методички використано `[Column(TypeName = "decimal(18, 2)")]`. Це також робочий варіант для SQL Server. `[Precision(10, 2)]` виражає той самий намір у більш незалежній від конкретної БД формі.

3. URL, маршрути та параметри

Адреса вебсторінки складається зі шляху та, за потреби, рядка запиту:

```
/Books/Details/7                шлях із параметром маршруту
/Books/Details?id=7             шлях + рядок запиту
/Books?searchString=Кобзар      пошуковий рядок запиту
```

Директива `@page`

У Razor Page директива `@page` робить файл кінцевою точкою, яка може приймати HTTP-запити.

```
@page "{id:int}"
@model BookCatalog.Pages.Books.DetailsModel
```

`{id:int}` означає: URL має містити параметр `id`, і він повинен бути цілим числом. Тому `/Books/Details/7` відповідає маршруту, а `/Books/Details/abc` — ні. Якщо маршрут не підходить, ASP.NET Core поверне **404 Not Found** ще до виклику `OnGetAsync`.

```
@page "{id:int?}"
```

Символ `?` робить параметр необов'язковим. У такому разі запит `/Books/Details` уже дійде до обробника, але код має сам вирішити, що робити з відсутнім `id`.

```
public async Task<IActionResult> OnGetAsync(int? id)
{
    if (id is null)
    {
        return NotFound();
    }

    Book = await _context.Book.FirstOrDefaultAsync(item => item.Id == id);

    return Book is null ? NotFound() : Page();
}
```

Тут є два різні випадки 404:

1. маршрут не відповів URL (`id` не є числом або його обов'язково немає);
2. маршрут підходить, але запису з таким `id` немає в БД.

4. Anchor Tag Helper і формування посилань

У Razor не потрібно вручну складати URL рядком.

```
<a asp-page="./Edit" asp-route-id="@item.Id">Редагувати</a>
```

Це **Anchor Tag Helper**. Він знає маршрут сторінки `Edit` і передає значення `item.Id`. Якщо в `Edit.cshtml` є `@page "{id:int}"`, результатом стане приблизно такий HTML:

```
<a href="/Books/Edit/7">Редагувати</a>
```

Якщо шаблону маршруту немає, буде створено посилання на кшталт `/Books/Edit?id=7`.

Перевага Tag Helpers: коли маршрут зміниться, код посилання не доведеться вручну переписувати у всіх сторінках.

5. GET, POST і зв'язування моделі

HTTP GET

GET-запит використовується для читання даних або відкриття сторінки. Наприклад, перехід до `/Books/Edit/7` викликає `OnGetAsync`, завантажує книгу з БД і показує форму з її значеннями.

HTTP POST

POST-запит використовується, коли користувач надсилає форму створення чи редагування. Значення з `<input>` переходять до C#-моделі.

```
[BindProperty]
public Book Book { get; set; } = default!;

public async Task<IActionResult> OnPostAsync()
{
    if (!ModelState.IsValid)
    {
        return Page();
    }

    _context.Attach(Book).State = EntityState.Modified;
    await _context.SaveChangesAsync();

    return RedirectToPage("./Index");
}
```

`[BindProperty]` дозволяє **model binding**: ASP.NET Core зіставляє імена полів форми (`Book.Title`, `Book.Price`) із властивостями об'єкта `Book`.

`ModelState.IsValid` має бути перевірено завжди. Якщо введені дані не пройшли правила `[Required]`, `[Range]` тощо, повертається та сама сторінка з повідомленнями про помилки.

Після успішного збереження застосовується шаблон **Post-Redirect-Get**:

```
POST /Books/Edit/7 → зберегти дані → RedirectToPage("./Index") → GET /Books
```

Це не дає браузеру випадково надіслати форму вдруге після оновлення сторінки.

Обережно з прив'язуванням

`[BindProperty] public Book Book` зручно використовувати в навчальній роботі. У більшому застосунку не варто бездумно дозволяти користувачеві заповнювати всі властивості сутності: так може виникнути **overposting** — надсилання полів, яких немає у формі, але які не повинен змінювати користувач. Типове рішення — окрема модель форми або явний перелік дозволених властивостей.

6. Оновлення та конфлікт паралельності

Після POST до сторінки редагування цей рядок повідомляє EF Core, що об'єкт слід оновити:

```
_context.Attach(Book).State = EntityState.Modified;
```

Під час `SaveChangesAsync()` EF Core формує SQL-команду `UPDATE`.

Іноді два користувачі відкривають один запис одночасно. Один користувач видаляє його, інший натискає «Зберегти». У такому разі EF Core може створити `DbUpdateConcurrencyException`.

```
catch (DbUpdateConcurrencyException)
{
    if (!BookExists(Book.Id))
    {
        return NotFound();
    }

    throw;
}
```

У цій роботі перевіряється простий сценарій: чи існує запис після невдалої спроби збереження. Реальні системи можуть додатково порівнювати версії запису та пропонувати користувачу вирішити конфлікт.

7. Пошук: LINQ і `IQueryable`

LINQ — синтаксис C#, який дозволяє описати вибірку, фільтрацію й сортування даних. Приклад пошуку за назвою:

```
public async Task OnGetAsync()
{
    IQueryable<Book> books = _context.Book;

    if (!string.IsNullOrWhiteSpace(SearchString))
    {
        books = books.Where(item => item.Title.Contains(SearchString));
    }

    Book = await books.ToListAsync();
}
```

Розбір коду:

- `_context.Book` представляє таблицю книг;
- `IQueryable<Book>` — ще не готовий список, а опис запиту до БД;
- `Where(...)` додає умову `WHERE`;

- `item => item.Title.Contains(SearchString)` — лямбда-вираз, тобто короткий запис функції «для кожної книги перевірити її назву»;
- `ToListAsync()` виконує запит і повертає фактичний список.

До `ToListAsync()` запит можна доповнювати. Це називають **відкладеним виконанням**.

```
var books = _context.Book.AsQueryable();

if (!string.IsNullOrWhiteSpace(SearchString))
    books = books.Where(book => book.Title.Contains(SearchString));

if (!string.IsNullOrWhiteSpace(BookCategory))
    books = books.Where(book => book.Category == BookCategory);

Book = await books.OrderBy(book => book.Title).ToListAsync();
```

EF Core намагається перекласти цей LINQ-запит у SQL і виконує фільтрацію в БД. Не варто викликати `ToListAsync()` до додавання всіх фільтрів, бо тоді зайві дані спочатку буде завантажено із БД у пам’ять застосунку.

Чутливість `Contains` до регістру залежить від налаштувань конкретної БД, зокрема її колації. Не слід припускати однакову поведінку SQL Server і SQLite.

8. Форма пошуку та `SupportsGet`

Пошук не змінює дані, тому форма використовує GET за замовчуванням:

```
<form>
    <label>
        Назва:
        <input asp-for="SearchString" />
    </label>
    <input type="submit" value="Шукати" />
</form>
```

Після надсилання адреса матиме вигляд:

```
/Books?SearchString=Кобзар
```

Щоб Razor Page отримала це значення під час GET-запиту, властивість має бути явно дозволена:

```
[BindProperty(SupportsGet = true)]
public string? SearchString { get; set; }
```

GET-параметри варто розглядати як введення користувача: перевіряти, обмежувати й не використовувати для небезпечних операцій на зразок видалення чи зміни даних.

9. Випадаючий список категорій

Для фільтра потрібні дві різні властивості:

```
public SelectList? Categories { get; set; }

[BindProperty(SupportsGet = true)]
public string? BookCategory { get; set; }
```

- `Categories` містить усі варіанти, які треба показати у списку.
- `BookCategory` зберігає один вибраний користувачем варіант.

```
IQueryable<string> categoryQuery = _context.Book
    .OrderBy(book => book.Category)
    .Select(book => book.Category);

Categories = new SelectList(await categoryQuery.Distinct().ToListAsync());
```

- `Select` вибирає тільки значення `Category`;
- `Distinct` прибирає повтори;
- `OrderBy` сортує список;
- `SelectList` — формат, який Tag Helper може використати для `<select>`.

```
<select asp-for="BookCategory" asp-items="Model.Categories">
    <option value="">Усі категорії</option>
</select>
```

`asp-for` зберігає вибране значення, а `asp-items` створює варіанти `<option>`.

10. Зміна моделі та міграції EF Core

Коли додається властивість, наприклад `Publisher`, змінюється C#-модель. Але таблиця в існуючій БД сама по собі не змінюється.

Правильна послідовність:

```
1. Змінити клас Book
2. Зібрати проєкт
3. Створити міграцію
4. Переглянути створену міграцію
5. Застосувати її до БД
6. Оновити сторінки Create, Edit, Details, Delete, Index і SeedData
```

Команди Package Manager Console:

```
Add-Migration AddPublisher
Update-Database
```

Або команди термінала:

```
dotnet ef migrations add AddPublisher
dotnet ef database update
```

- `Add-Migration` **не змінює БД**: вона створює C#-файли з описом змін.
- `Update-Database` застосовує ще не застосовані міграції до БД.
- Таблиця `__EFMigrationsHistory` зберігає перелік уже застосованих міграцій.

> EF Core не виконує повну автоматичну перевірку відповідності моделі й схеми під час запуску застосунку. Якщо код звернеться до нового стовпця, якого ще немає у БД, БД зазвичай поверне помилку на кшталт `Invalid column name`. Саме тому міграції треба створювати й застосовувати свідомо.

Як безпечно додати нове обов'язкове поле

Якщо в таблиці вже є записи, додати `NOT NULL`-стовпець без значення за замовчуванням може бути неможливо. Типовий навчальний варіант:

1. тимчасово зробити нове поле nullable;
2. заповнити старі записи;
3. наступною міграцією зробити поле обов'язковим.

Або встановити коректне значення за замовчуванням у міграції. Видаляти БД можна лише для локальних тестових даних, не для реальної системи.

11. Валідація: клієнт і сервер

Валідація має працювати у двох місцях:

```
Браузер → швидке повідомлення користувачеві
Сервер  → обов'язкова остаточна перевірка перед збереженням
```

Правила в моделі

```
[Required(ErrorMessage = "Вкажіть категорію")]
[StringLength(60)]
public string Category { get; set; } = string.Empty;

[Range(0.01, 1_000_000)]
public decimal Price { get; set; }
```

`[Required]`, `[StringLength]`, `[Range]` і `[RegularExpression]` — правила валідації. Натомість `[Display]` і `[DataType]` описують назву або спосіб подання, але самі по собі не перевіряють коректність даних.

Відображення помилок у Razor

```
<div asp-validation-summary="ModelOnly" class="text-danger"></div>

<label asp-for="Book.Title"></label>
<input asp-for="Book.Title" class="form-control" />
<span asp-validation-for="Book.Title" class="text-danger"></span>

@section Scripts {
    @{ await Html.RenderPartialAsync("_ValidationScriptsPartial"); }
}
```

- `asp-validation-summary` показує загальні помилки моделі;
- `asp-validation-for` показує помилку конкретного поля;
- `_ValidationScriptsPartial` підключає JavaScript для клієнтської валідації.

Навіть якщо JavaScript вимкнено або змінено дані запиту вручну, серверна перевірка через `ModelState.IsValid` залишається обов'язковою.

Регулярні вирази: застосовуйте обережно

Регулярний вираз може бути корисним, але надмірно вузьке правило відхиляє нормальні дані. Наприклад, вираз, який дозволяє лише англійські літери, не підходить для українських назв, апострофів, дефісів або назв товарів із цифрами.

Початківцю краще почати з `[Required]`, `[StringLength]` і `[Range]`, а регулярний вираз застосовувати лише коли формат справді відомий: номер телефону, код товару, номер групи тощо.

12. Типові помилки

Симптом	Причина	Що зробити
`Invalid column name ...`	Змінено модель, але не застосовано міграцію	`Add-Migration ...`, потім `Update-Database`.
Поле є в БД, але його немає у формі	Змінено лише модель	Оновити Create, Edit, Details, Delete та Index.
Фільтр не працює	Немає `SupportsGet = true`, відмінні назви властивостей	Перевірити `SearchString`, `BookCategory` і `asp-for`.
У фільтрі дублюються категорії	Не застосовано `Distinct()`	Побудувати список категорій через `Distinct()`.
Валідація не показується до надсилання форми	Немає `_ValidationScriptsPartial` або Tag Helpers	Перевірити секцію `Scripts`, `asp-for`, `asp-validation-for`.
Дані все одно потрапляють на сервер із помилками	Це нормально без JavaScript	Сервер повинен перевірити `ModelState.IsValid`.
`/Books/Details/abc` повертає 404	Маршрут вимагає `{id:int}`	Використати числовий ідентифікатор.
Помилка `DbUpdateConcurrencyException`	Запис змінили або видалили паралельно	Перевірити, чи існує запис, і пояснити користувачу ситуацію.

13. Мінімальний чекліст перед здаванням

- [ ] Назви полів у UI зрозумілі для обраної предметної області.
- [ ] Посилання Edit, Details і Delete працюють із `id` у маршруті.
- [ ] Пошук за текстом виконується через GET і його значення видно в URL.
- [ ] Випадаючий список категорій не містить повторів.
- [ ] Пошук за текстом і категорією може працювати одночасно.
- [ ] Додано власне нове поле, а не обов'язково саме `Rating`.
- [ ] Для зміни моделі створено й застосовано міграцію.
- [ ] Нове поле показане в усіх потрібних CRUD-сторінках і в seed-даних.
- [ ] Перевірено хоча б три помилкові значення форми.
- [ ] Ви можете пояснити різницю між `IQueryable` і готовим `List`, GET і POST, `[DataType]` і правилом валідації.

14. Головний зв'язок тем лабораторної роботи

```
URL і форма
        ↓ model binding
```

```
PageModel
        ↓ LINQ / EF Core
DbContext
        ↓ SQL
База даних

Зміна моделі
        ↓ migration
Оновлення схеми БД

Data Annotations
        ↓
UI-підписи + клієнтська і серверна валідація + частина обмежень схеми
```

Якщо цей ланцюжок зрозумілий, можна самостійно додавати поля, фільтри та правила до будь-якого Razor Pages-каталогу, а не просто відтворювати код із прикладу.